

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



**FACULTAD DE INGENIERÍA EN
SISTEMAS, ELECTRÓNICA E INDUSTRIAL**

CARRERA DE SOFTWARE

Título: Tarea 4 — Laboratorio Gestor de impresiones

Alumno:

Cunalata Mendoza Damian Alexander

Ciclo Académico y Paralelo: Tercero “B”

Módulo y Docente: Estructura de Datos – Ing. José Caiza.

JULIO 2026 - DICIEMBRE 2026

1. Ejercicio

El laboratorio indica construir un gestor de impresiones que pueda administrar dos grupos de documentos, los que esperan turno para salir impresos y los que ya salieron.

Mientras un documento espera lo justo es conservar el orden en que llegó. Sería incorrecto que un archivo registrado hace un segundo se imprima antes que otro que lleva varios minutos en la fila. En cambio, cuando se quiere deshacer una impresión lo natural es recuperar la última que salió, igual que funciona el botón de deshacer de cualquier programa, nadie espera que deshacer le devuelva el primer cambio que hizo en el día.

Esa diferencia de criterio es exactamente la que obliga a usar dos estructuras distintas. Los documentos pendientes se guardan en una cola, donde el primero en entrar es el primero en salir. Los documentos ya impresos se apilan en un historial, donde el último en entrar es el primero en salir. La consigna define cuatro acciones y sugiere para cada una la operación que debe emplearse.

Tabla 1. Acciones solicitadas por el laboratorio y operación sugerida para cada una.

Acción	Estructura	Operación sugerida	Resultado
Registrar documento	pendientes	<code>offerLast(nombre)</code>	Entra al final
Imprimir siguiente	pendientes	<code>pollFirst()</code>	Sale el más antiguo
Guardar impresión	historial	<code>push(nombre)</code>	Queda en la cima
Recuperar última	historial y pendientes	<code>pop()</code> y <code>addFirst()</code>	Vuelve al frente

Junto a esas cuatro acciones, el enunciado impone cuatro condiciones adicionales, validar que las estructuras no estén vacías antes de retirar un elemento, probar al menos seis operaciones combinadas, declarar ambas variables como `Deque<String>` respetando el contrato de cada estructura, y entregar el código, la salida del programa y la traza manual.

2. Solución propuesta

Se construyó un TDA llamado `GestorImpresiones` que encapsula las dos estructuras y expone únicamente las operaciones necesarias para trabajar con ellas. Quien usa el gestor no toca la cola ni la pila de forma directa: solo llama métodos públicos con nombres que describen acciones del mundo real, como registrar un documento o imprimir el siguiente. Si mañana se decidiera cambiar `ArrayDeque` por otra implementación, el programa que usa el gestor no tendría que modificarse.

El programa se resolvió con dos clases, una que contiene el TDA y otra que lo somete a prueba.

Tabla 2. Clases que componen el programa y responsabilidad de cada una.

Clase	Responsabilidad
<code>GestorImpresiones</code>	TDA que administra la cola de pendientes y la pila de historial. Contiene las dos estructuras y todas las operaciones que actúan sobre ellas.
<code>DemoGestorImpresiones</code>	Banco de pruebas. Ejecuta una secuencia fija de once operaciones combinadas y muestra el estado de ambas estructuras después de cada una.

3. Relación entre requisitos y código

Antes de revisar el código paso a paso conviene ubicar dónde se cumple cada exigencia del enunciado, de modo que ninguna quede sin evidencia.

Tabla 3. Correspondencia entre cada requisito del laboratorio y su aplicación en el código.

Requisito	Aplicación
Cola de documentos pendientes	La variable pendientes, manipulada con offerLast y pollFirst.
Pila de documentos impresos	La variable historial, manipulada con push y pop.
Declarar ambas como Deque<String>	Las dos usan ese tipo; la implementación concreta es ArrayDeque<String>.
Respetar el contrato de cada estructura	Cada variable admite solo los métodos de su disciplina. La única excepción es addFirst, justificada en el paso 5.
Validar estructuras vacías antes de retirar	El método isEmpty() se consulta antes de cada pollFirst() y de cada pop().
Probar al menos seis operaciones combinadas	DemoGestorImpresiones ejecuta once pasos, incluidos tres casos de rechazo.
Entregar código, salida y traza manual	Código en las secciones 4 a 8; prueba de escritorio y salida real en la sección 13.

4. Paso 1: importar las clases necesarias

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
```

Deque es la interfaz que declara las operaciones disponibles en ambos extremos de una estructura lineal. Es el tipo que exige la consigna y el que permite que una misma declaración sirva tanto para la cola como para la pila.

ArrayDeque es la implementación concreta que se instancia. Iterator se emplea únicamente para recorrer las estructuras cuando hay que mostrar su estado en pantalla; se trata de un recorrido de lectura que no retira ni agrega elementos, por lo que no vulnera el contrato de ninguna de las dos estructuras.

No se importa Scanner porque este programa no solicita datos por teclado. El banco de pruebas ejecuta una secuencia fija de operaciones, lo que permite que la salida sea siempre la misma y pueda compararse contra la prueba de escritorio.

5. Paso 2: crear las dos estructuras

```
public class GestorImpresiones {

    // pendientes se usa solo como COLA: offerLast, pollFirst, isEmpty, size.
    private final Deque<String> pendientes;
    // historial se usa solo como PILA: push, pop, isEmpty, size.
    private final Deque<String> historial;

    public GestorImpresiones() {
        pendientes = new ArrayDeque<>();
        historial = new ArrayDeque<>();
    }
}
```

Las dos variables se declaran con el tipo Deque<String> y se instancian con ArrayDeque, tal como pide el enunciado. La palabra private impide que el código externo manipule las estructuras saltándose los métodos del TDA, y final impide que se les asigne otra estructura después de construido el gestor; no impide, en cambio, agregar o retirar elementos.

La elección de ArrayDeque no es arbitraria. Stack es una clase heredada de las primeras versiones de Java, está sincronizada y arrastra un costo que aquí no aporta nada. LinkedList funcionaría, pero guarda cada elemento en un nodo independiente con referencias al anterior y al siguiente, lo que ocupa más memoria y dispersa los datos. ArrayDeque trabaja sobre un arreglo circular y resuelve las inserciones y retiros en ambos extremos en tiempo constante, que es justo lo que necesitan una cola y una pila.

Vale anotar una restricción propia de ArrayDeque: no admite el valor null. Por eso el registro valida el nombre antes de encolarlo, como se ve en el paso siguiente.

6. Paso 3: registrar documentos

```
public boolean registrarDocumento(String nombre) {
    if (nombre == null || nombre.trim().isEmpty()) {
        return false;
    }
    pendientes.offerLast(nombre);
    return true;
}
```

El método valida antes de actuar. Descarta el valor null, que ArrayDeque no acepta, y descarta también los nombres formados solo por espacios, porque un documento sin nombre no podría identificarse después en la cola. Solo cuando el nombre es válido se llama a offerLast, que coloca el documento al final de la fila.

El método devuelve un valor booleano en lugar de no devolver nada. Así quien lo llama se entera de si el registro se concretó y puede reaccionar, en vez de suponer que todo salió bien.

7. Paso 4: imprimir y archivar

```
public String imprimirSiguiente() {
    if (pendientes.isEmpty()) {
        return null;
    }
    String actual = pendientes.pollFirst();
    guardarImpresion(actual);
    return actual;
}

private void guardarImpresion(String nombre) {
    historial.push(nombre);
}
```

Lo primero que hace el método es preguntar si hay algo por imprimir. Si la cola está vacía devuelve null y no modifica nada; esta es la validación que exige el enunciado y evita que pollFirst se ejecute sobre una estructura sin elementos.

Cuando sí hay documentos, pollFirst retira el que está al frente, que por construcción es el más antiguo, y acto seguido el método lo archiva en el historial. Esas dos acciones ocurren juntas de manera intencional.

Por esa razón guardarImpresion se declaró privado. La consigna lo presenta como una acción aparte, pero exponerlo como método público permitiría archivar un documento que nunca se imprimió, o imprimir uno sin dejar registro. Imprimir y archivar son una sola transacción, y encapsularlas juntas garantiza que el historial refleje siempre lo que realmente salió por la impresora.

8. Paso 5: recuperar la última impresión

```
public String recuperarUltima() {  
    if (historial.isEmpty()) {  
        return null;  
    }  
    String ultimo = historial.pop();  
    // addFirst es la unica excepcion al uso de pendientes como cola pura:  
    // el documento ya espero su turno una vez, asi que vuelve al frente  
    // en lugar de hacer fila de nuevo al final.  
    pendientes.addFirst(ultimo);  
    return ultimo;  
}
```

El método aplica la misma validación que el anterior, esta vez sobre el historial: si no hay ninguna impresión archivada no hay nada que recuperar y se devuelve null sin tocar las estructuras.

Si el historial tiene elementos, pop retira el que está en la cima, que es la impresión más reciente, y addFirst lo devuelve al frente de la cola de pendientes. Esta es la única línea de todo el programa que inserta un elemento por el frente de la cola y, por lo tanto, la única excepción al contrato FIFO.

La excepción está justificada y no es un descuido. El documento ya hizo fila una vez y ya fue impreso; mandarlo al final de la cola lo obligaría a esperar dos veces por una situación que no provocó. Devolverlo al frente respeta el turno que ya había ganado. Es una decisión de negocio, y por eso quedó documentada con un comentario en el propio código.

9. Tipos de datos utilizados

El programa combina tipos primitivos, que guardan un valor directamente, y tipos de referencia, que guardan la dirección de un objeto creado en memoria. La distinción importa porque las dos estructuras del TDA no almacenan documentos, sino referencias a las cadenas que los nombran.

Tabla 4. Tipos de datos empleados en el programa, clasificados por categoría.

Tipo	Categoría	Variables y usos
boolean	Primitivo	Valor devuelto por registrarDocumento, estaVaciaPendientes y estaVacioHistorial.
int	Primitivo	Valor devuelto por tamañoPendientes y tamañoHistorial; parámetro numero del método paso.
String	Referencia	nombre del documento, actual, ultimo y las descripciones de cada paso.
Deque<String>	Referencia (interfaz)	Tipo declarado de pendientes y de historial.
ArrayDeque<String>	Referencia (implementación)	Objetos concretos creados en el constructor del gestor.
Iterator<String>	Referencia	Recorrido de lectura empleado para mostrar el estado.

10. Diagrama de clases

El diagrama recoge las dos clases del programa con sus atributos y métodos. El signo menos indica miembros privados y el signo más, miembros públicos.

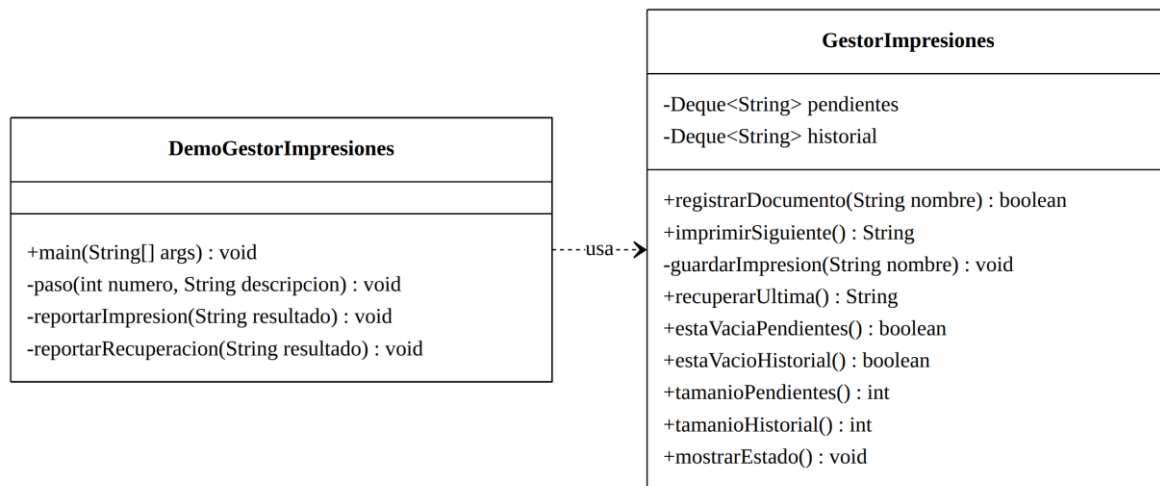


Figura 1. Diagrama de clases del gestor de impresiones.

Se observa que los dos atributos del gestor son privados y que los únicos métodos privados son guardarImpresion y los auxiliares de presentación. Todo lo demás es la interfaz pública del TDA, es decir, lo único que necesita conocer quien lo use.

11. Especificación del TDA

La especificación describe qué hace el TDA sin comprometerse con cómo lo hace. Es el contrato que se pacta con quien lo usa y el que permite cambiar la implementación más adelante sin romper nada.

Estado

- pendientes: cola de documentos en espera. Los elementos entran por el final y salen por el frente.
- historial: pila de documentos ya impresos. Los elementos entran y salen por la cima.
- documento: cadena de texto que identifica a un archivo dentro del gestor.

Ambas estructuras nacen vacías al construir el gestor y están encapsuladas, de manera que solo pueden modificarse a través de las operaciones descritas a continuación.

Operaciones

Tabla 5. Operaciones que expone el TDA y resultado de cada una.

Operación	Resultado
registrarDocumento()	Agrega un documento al final de la cola de pendientes si el nombre es válido. Devuelve verdadero o falso según se haya registrado.
imprimirSiguiente()	Retira el documento más antiguo de pendientes y lo archiva en el historial. Devuelve nulo si no hay documentos en espera.
guardarImpresion()	Coloca un documento impreso en la cima del historial. Es interna al TDA y se ejecuta siempre junto a la impresión.
recuperarUltima()	Retira la última impresión del historial y la devuelve al frente de pendientes. Devuelve nulo si el historial está vacío.

Operación	Resultado
estaVaciaPendientes()	Indica si no quedan documentos en espera.
estaVacioHistorial()	Indica si no hay impresiones archivadas.
tamanoPendientes()	Devuelve cuántos documentos esperan turno.
tamanoHistorial()	Devuelve cuántas impresiones están archivadas.
mostrarEstado()	Presenta el contenido de la cola y de la pila sin modificarlas.

Invariantes

Son las reglas que el TDA cumple en todo momento, antes y después de cualquier operación.

1. El documento que sale de pendientes es siempre el más antiguo de los que esperan.
2. El documento que sale del historial es siempre el último que se imprimió.
3. Solamente se puede imprimir si existe al menos un documento pendiente.
4. Solamente se puede recuperar si el historial contiene al menos una impresión.
5. Imprimir un documento siempre lo archiva en el historial; las dos acciones nunca ocurren por separado.
6. Un documento recuperado vuelve al frente de pendientes y no al final, porque ya esperó su turno.
7. Ningún documento se pierde: en todo momento está esperando en la cola o archivado en la pila.
8. Un nombre nulo o formado solo por espacios nunca se registra.
9. El número de documentos en cada estructura nunca es negativo.

12. Pseudocódigo

El siguiente pseudocódigo resume la lógica del TDA sin detalles del lenguaje, de modo que pueda trasladarse a Java, a C++ o a cualquier otro entorno que ofrezca una cola y una pila.

```

INICIO
  crear gestor con pendientes vacia e historial vacio

  OPERACION registrar(nombre)
    SI nombre es nulo O nombre esta en blanco ENTONCES
      devolver FALSO
    SINO
      encolar nombre al final de pendientes
      devolver VERDADERO
    FIN SI

  OPERACION imprimir()
    SI pendientes esta vacia ENTONCES
      devolver NULO
    SINO
      documento <- retirar del frente de pendientes
      apilar documento en la cima de historial
      devolver documento
    FIN SI

  OPERACION recuperar()
    SI historial esta vacio ENTONCES
      devolver NULO
    SINO
      documento <- retirar de la cima de historial
      insertar documento al frente de pendientes
      devolver documento
    FIN SI

  OPERACION mostrarEstado()
    recorrer pendientes desde el frente hasta el final
    recorrer historial desde la cima hasta el fondo
FIN

```

13. Prueba de escritorio

La prueba de escritorio recorre paso a paso la secuencia de operaciones del banco de pruebas y registra cómo queda cada estructura después de cada una. El enunciado exige al menos seis operaciones combinadas; se ejecutaron once para cubrir también los casos de rechazo.

La cola se lee de izquierda a derecha desde el frente hacia el final, y la pila se lee de izquierda a derecha desde la cima hacia el fondo.

Tabla 6. Prueba de escritorio: estado de la cola y de la pila tras cada operación.

Paso	Operación	Pendientes (frente a final)	Historial (cima a fondo)	Resultado
0	Estado inicial	[]	[]	Ambas estructuras vacías
1	Imprimir con cola vacía	[]	[]	Rechazado: no hay pendientes
2	Recuperar con historial vacío	[]	[]	Rechazado: no hay impresiones
3	Registrar nombre en blanco	[]	[]	Rechazado: nombre inválido
4	Registrar Informe.pdf	[Informe.pdf]	[]	Entra al final

Paso	Operación	Pendientes (frente a final)	Historial (cima a fondo)	Resultado
5	Registrar Tarea.docx	[Informe.pdf, Tarea.docx]	[]	Entra al final
6	Registrar Foto.png	[Informe.pdf, Tarea.docx, Foto.png]	[]	Entra al final
7	Imprimir siguiente	[Tarea.docx, Foto.png]	[Informe.pdf]	Sale el más antiguo y se archiva
8	Imprimir siguiente	[Foto.png]	[Tarea.docx, Informe.pdf]	Se apila sobre el anterior
9	Recuperar última impresión	[Tarea.docx, Foto.png]	[Informe.pdf]	Vuelve al frente, no al final
10	Imprimir siguiente	[Foto.png]	[Tarea.docx, Informe.pdf]	El recuperado sale primero
11	Imprimir siguiente	[]	[Foto.png, Tarea.docx, Informe.pdf]	La cola queda vacía

Los pasos 1, 2 y 3 comprueban las validaciones antes de que exista cualquier dato, que es cuando un programa mal escrito fallaría. Los pasos 7 y 8 muestran el comportamiento FIFO de la cola junto al comportamiento LIFO de la pila operando a la vez. Los pasos 9 y 10, leídos en conjunto, son la evidencia de que la excepción documentada en el paso 5 del código funciona como se prometió.

Conviene notar que entre el paso 4 y el paso 11 la suma de documentos en ambas estructuras nunca disminuye: los archivos se mueven de una a otra, pero ninguno desaparece. Eso confirma el invariante número siete.

Salida real del programa

La ejecución del programa produce la siguiente salida, que coincide con la prueba de escritorio anterior en todos los pasos, además, esta salida se encuentra en el repositorio.

```

=== Paso 1: Imprimir con la cola de pendientes vacia ===
No hay documentos pendientes
Pendientes (frente -> final): []
Historial (cima -> fondo): []

=== Paso 2: Recuperar con el historial vacio ===
No hay impresiones para recuperar
Pendientes (frente -> final): []
Historial (cima -> fondo): []

=== Paso 3: Registrar nombre en blanco ===
Registrado: false
Pendientes (frente -> final): []
Historial (cima -> fondo): []

=== Paso 4: Registrar Informe.pdf ===
Registrado: true
Pendientes (frente -> final): [Informe.pdf]
Historial (cima -> fondo): []

=== Paso 5: Registrar Tarea.docx ===
Registrado: true
Pendientes (frente -> final): [Informe.pdf, Tarea.docx]
Historial (cima -> fondo): []

=== Paso 6: Registrar Foto.png ===
Registrado: true

```

```
Pendientes (frente -> final): [Informe.pdf, Tarea.docx, Foto.png]
Historial (cima -> fondo): []

=== Paso 7: Imprimir siguiente ===
Se imprimio: Informe.pdf
Pendientes (frente -> final): [Tarea.docx, Foto.png]
Historial (cima -> fondo): [Informe.pdf]

=== Paso 8: Imprimir siguiente ===
Se imprimio: Tarea.docx
Pendientes (frente -> final): [Foto.png]
Historial (cima -> fondo): [Tarea.docx, Informe.pdf]

=== Paso 9: Recuperar ultima impresion ===
Se recupero: Tarea.docx
Pendientes (frente -> final): [Tarea.docx, Foto.png]
Historial (cima -> fondo): [Informe.pdf]

=== Paso 10: Imprimir siguiente ===
Se imprimio: Tarea.docx
Pendientes (frente -> final): [Foto.png]
Historial (cima -> fondo): [Tarea.docx, Informe.pdf]

=== Paso 11: Imprimir siguiente ===
Se imprimio: Foto.png
Pendientes (frente -> final): []
Historial (cima -> fondo): [Foto.png, Tarea.docx, Informe.pdf]
```

14. Anexos

Se adjunta el enlace del repositorio donde se podrá observar el código realizado junto con su respectivo Readme.md y la salida.txt, además de capturas de ejecución del código:

https://github.com/Damienq7w/Estructura_Datos_T4.-Laboratorio-Gestor-de-impresiones